

ECE 1387

Lecture #4

October 3, 2025

- Routing cont'd
- Placement

How is AI
going?

Overall algorithm

- 1) Route all signals allowing shorts
- 2) while (shared resources exist ^(shorts) ~~not exhausted~~) {
 loop over all nets i {
 rip up routing for net i
 Route(i) $\leftarrow$ Source i
 loop until all loads t_{ij} on net i ^{are routed} {
 init priority queue (expansion list)
 to everything in Route(i) at
 Cost 0
 expansion target. } }

loop until t_{ij} is found {
pop lowest cost node m from
PRQ
if m is t_{ij} break

loop over nodes n connected to m in
RRG {

if n already
visited
continue

add n to PRQ at cost of
 $C_n + \text{PRQ}(\text{cost}(m))$

cost m
in PRQ

}
back trace from t_{ij} to $\text{Route}(i)$
→ add trace to $\text{Route}(i)$

adjust P_n 's accordingly

} // until all loads t_{ij} hit

} // until all signals raised

adjust P_{fac} , h_{fac} // increase these
update h_n 's accordingly

} // outer while loop

Each iteration of while loop
is one Nair iterations
(routes all signals)

A Machine Learning Approach for Predicting the Difficulty of FPGA Routing Problems

Andrew David Gunter
Department of Electrical and
Computer Engineering
University of British Columbia
Vancouver, British Columbia
Email: agunter@ece.ubc.ca

Steven J.E. Wilton
Department of Electrical and
Computer Engineering
University of British Columbia
Vancouver, British Columbia
Email: stevew@ece.ubc.ca

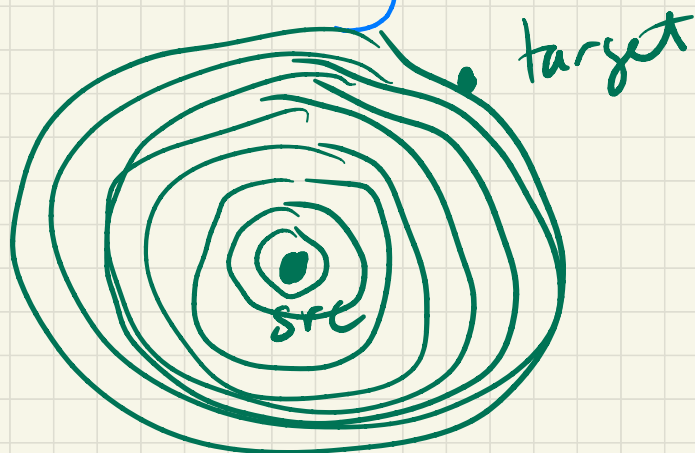
Abstract—In this paper, we present a Machine Learning (ML) Mixture of Experts (MoE) technique to predict the number of iterations needed for a Pathfinder-based FPGA router to complete a routing problem. Given a placed circuit, our technique uses features gathered on each routing iteration to predict if the circuit is routable and how many more iterations will be required to successfully route the circuit. This enables early exit for routing problems which are unlikely to be completed in a target number of iterations. Such early exit may help to achieve a successful route within tractable time by allowing the user to quickly retry the circuit compilation with a different random seed, a modified circuit design, or a different FPGA. We demonstrate our predictor in the VTR 8 framework; compared to VTR's predictor, our ML predictor incurs lower prediction errors on the Koios Deep Learning and Titan23 benchmark suites. Based on our tests, equipping VTR with our ML predictor would reduce time wasted on unroutable designs by 31% while also allowing 28% more routable designs to be completed.

or modified design). There are many papers showing how post-placement circuit information can be used to predict a circuit's routability [1]–[6]. Unfortunately, this post-placement information is limited and can lead to inaccurate predictions as routability is intricately tied to the detailed routing architecture, which is difficult to model [7], [8]. Even if these approaches estimate routability, they typically do not relate the estimate to the run-time required by the router. Some circuits may be routable but require more time than the user is willing to tolerate.

In this paper, we present a Machine Learning (ML) Mixture of Experts (MoE) technique to predict the number of iterations needed for a Pathfinder-based FPGA router [9] to complete a routing problem. Our technique uses information obtained *during* the routing process, allowing decisions to be based on the actual interaction between the FPGA routing architecture and

A* routing

(all commercial routers)



maze routing

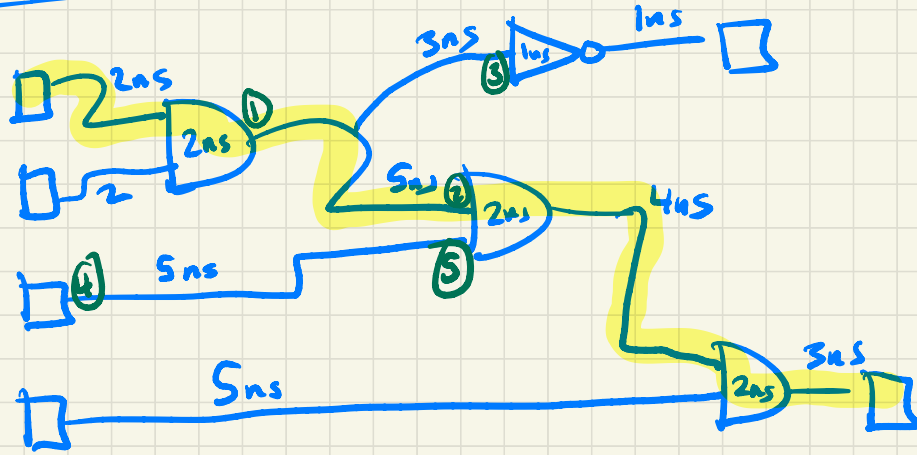
A* routing

→ needed for runtime

How to make algorithm give faster resources to connections that need them?

- ① Determine which conns are timing critical
- ② For critical connections, want Path Finder to pay more attention to delay (vs congestion)

Timing Analysis



$$D_{1,2} = 20 \text{ ns}$$

$$D_{1,3} = 9 \text{ ns}$$

$$D_{4,5} = 16 \text{ ns}$$

$$A_{1,2} = 1$$

$$A_{1,3} = 9/20$$

$$A_{4,5} = 4/5$$

crit path delay is $2 + 2 + 5 + 2 + 4 + 2 + 3$
 $= 20 \text{ ns} \equiv D_{\text{max}}$

$D_{i,j} \equiv$ longest delay path thru pin i to j

Finally define $A_{ij} = \frac{D_{ij}}{D_{\max}}$

A_{ij} close to 1 $\Rightarrow$ conn $i \rightarrow j$
on or close to crit path

$A_{ij} \ll 1 \Rightarrow$ conn $i \rightarrow j$
not critical, so it's ok to
slow down.

New total cost of a node n when routing a connection from pin $i \rightarrow j$

$$\text{Cost}_n = A_{ij} \cdot d_n + (1 - A_{ij}) \cdot C_n$$

(
from last
week

Critical conns are routed
with greater emphasis on
delay

How to set A_{ij} initially?

- Need D_{ij} 's and D_{max} to compute these.
 - So...
 - could estimate delays from placement.
 - set all A_{ij} s to be 0.5 initially
 - set all A_{ij} s to be 1 initially
 - get high speed init. routing.
- have best-possible delays initially

$A_{i,j}$'s become stale why?

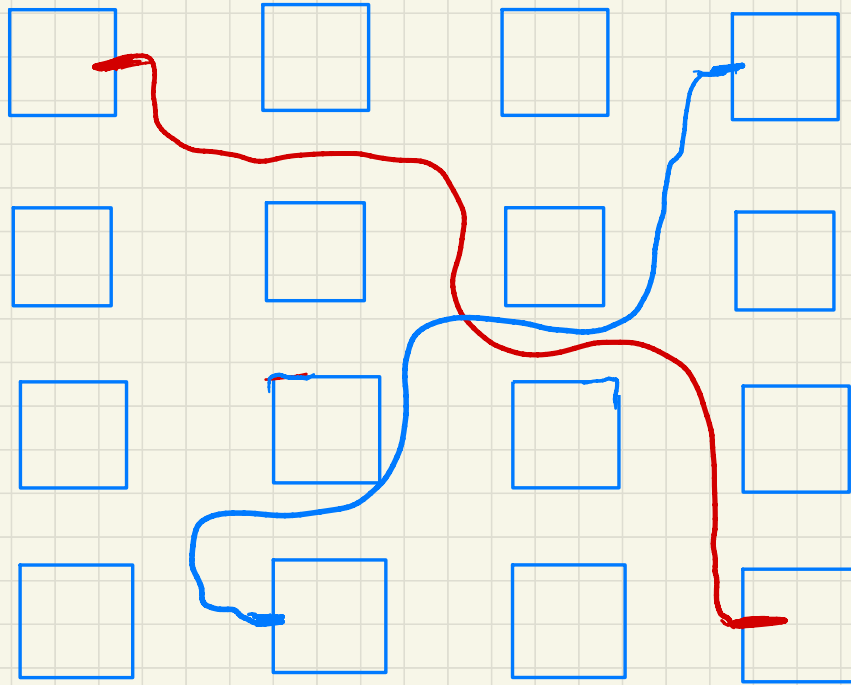
- As signals are ripped up and rerouted
- So need to periodically invoke timing analysis to get refresh $A_{i,j}$ values.

Another style of routing

Global Routing

- Key issue in CAD is how entire CAD flow is broken up into subtasks
- Pathfinder, maze routing solve entire routing problem
- Before this, one idea was break routing into two sub-tasks
 - 1) global routing
 - 2) detailed routing

Global routing. just decide which channels to use (not indiv. wire segments)



- Detailed routing chooses individual segments, and connections thru switch blocks

↳ honouring global routing

for FPGAs, global routing would say try not to exceed using

70% of W in any channel

↳ Otherwise detailed routing may fail.

Placement: step before routing

Problem definition

Given: 1) IC media (FPGA, standard cell)

2) netlist of cells and connections between them

Find: positions of cells

So as to minimize:

- wirelength (estimated)
- congestion
- area of layout
- critical path delay
- power

Complexity: If have n cells to place,
how many placements? at least
 $n!$

First order
goal of
any placer

giant search
space!

Naive algorithm:

for ~~all~~^{some} placements {
- route or global route
- save best
}

↳ will take an too long...

Cost function: a function we

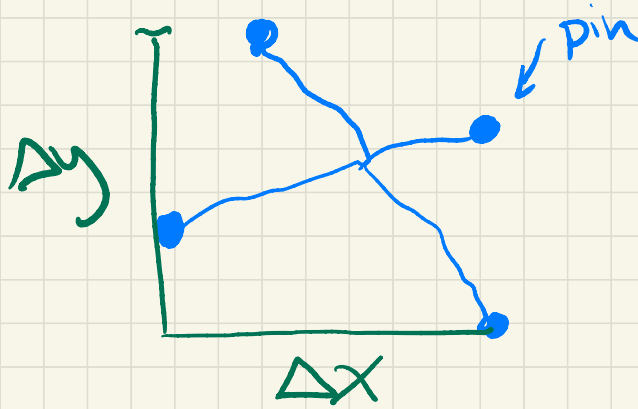
design or choose to measure
the "goodness" of a placement.

→ want this to be quick to
compute.

Most favourite wirelength estimation:

- Half-perimeter bounding box wirelength

"HPWL"



half-perim WL =

$$\underline{\Delta x} + \underline{\Delta y}$$

$$\text{Placement cost} = \sum_{i \in \text{nets}} \text{HPWL}(i) \quad \left. \vphantom{\sum_{i \in \text{nets}}} \right\} \begin{array}{l} \text{very} \\ \text{widely} \\ \text{used} \end{array}$$

